

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input, output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

```
stage01/  
  smartcare_v01.py  
  comparison.md  
  reflection.md  
  ai_usage.md
```

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

Accordingly, the data stored would include patient name, practitioner name, appointment time, and patient email address.

What functions might be useful?

- `type()`
- `print()`
- `input()`

What could go wrong?

If there were a broken rule in the programming language, the most common error complaint would be a syntax error; another could also possibly be an exception.

What requirements are unclear?

In what way should the information be presented in general, and what specific data types are allowed or to be used?

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

.....

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

```
    appointments.append(appointment)
```

```
def display_appointments():
```

```
    if not appointments:
```

```
        print("No appointments recorded.")
```

```
        return
```

```
    for appointment in appointments:
```

```
        print(f"Patient: {appointment['patient']} | Practitioner:
```

```
{appointment['practitioner']} | Time: {appointment['time']}")
```

```
print("Welcome to SmartCare: The Clinical Appointment Booking System!")
```

```
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
```

```
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
```

`display_appointments()`

^ Now, run both programs , and identify at least five limitations.

Basic One	Advanced One
Repeated variable names	Repeated functions
Repeated print functions	No validations
No validations	No data type conversions.
No data type conversions	No exception errors
Scaling is difficult	Data vulnerability

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

🧩 What the code *usually* does in an introductory appointment-booking function

Core takeaway: It collects user input (name, date, time), stores it in a simple structure (list, dictionary, or file), and prints a confirmation.

Most beginner appointment-booking functions follow this pattern:

- Ask the user for details such as **patient name**, **appointment date**, and **appointment time**.
- Validate the input in a basic way (e.g., checking if fields are empty).
- Store the appointment in a simple data structure:
 - a dictionary like {"name": ..., "date": ..., "time": ...}
 - or append it to a list such as `appointments.append(...)`
- Print a confirmation message such as “Appointment booked.”

If your version writes to a file, it likely:

- Opens a text file in append mode.
- Writes each appointment as a line.
- Closes the file.

⚠️ **Three limitations commonly found in beginner appointment-booking functions**

1. **No validation of date/time format** The function may accept impossible dates (e.g., 32/15/2026) or times (e.g., 99:99).
2. **No conflict checking** It probably doesn’t check whether the time slot is already booked.

3. **No error handling for file operations or bad input** If the file is missing or the user enters invalid data, the program may crash.

Improvements you can make without rewriting the whole application

1. **Add input validation** Use `datetime.strptime()` to ensure dates and times follow a valid format.
2. **Add appointment-conflict detection** Before saving, scan existing appointments to ensure the slot is free.
3. **Add basic error handling** Wrap file operations and input parsing in try/except blocks to prevent crashes and give helpful feedback.

Two questions to test your understanding

1. Why is it important to validate the date and time before saving an appointment?
2. What is one advantage of storing appointments in a list of dictionaries instead of a plain text file?

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
def book_appointment():
    # Ask the user for basic details
    patient = input("Enter patient name: ")
    practitioner = input("Enter practitioner name: ")
    time = input("Enter appointment time (e.g., 10:30 AM): ")

    # Store the appointment in a dictionary
    appointment = {
        "patient": patient,
        "practitioner": practitioner,
        "time": time
    }

    # Print confirmation
    print("\nAppointment booked:")
    print(f"Patient: {appointment['patient']}")
    print(f"Practitioner: {appointment['practitioner']}")
    print(f"Time: {appointment['time']}")

    return appointment
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Yes	Yes
Runs successfully?	Yes	Yes
Uses only required features?	Yes	Yes

Adds assumptions?	Yes	Yes
Handles errors?	No	No
Could I explain it?	Yes	Yes

Part F - Verify Behaviour

- Normal appointment
- Blank patient name
- Two appointments for the same practitioner/time
- Strange input such as patient_name=None or appointment_time=None

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

Part H - Reflection (150-250 words)

What did you build before using AI?

What did AI help you understand?

Did AI make assumptions?

How did you verify the AI output?

What engineering work remained for you?

The AI served as an interactive tutor throughout the development process, helping me break down complex programming concepts and iterate on potential functions that could be integrated into the system. By explaining how different components might behave and offering alternative approaches, it allowed me to explore a wider range of possibilities than I might have considered on my own. Once those foundational ideas were clarified, the next stage of the work would shift back to me. I would need to take the AI-generated insights and translate them into concrete programming objects, refining their structure and ensuring they aligned with the overall design of the project. From there, the focus would become hands-on testing—verifying that each object performed as expected, adjusting the logic where necessary, and gradually building toward a stable, functional codebase. In this way, the AI acted as a catalyst, while the final implementation and validation remained my responsibility.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.